

Contents

Task 1	1
Download Package.....	1
Registration	1
Pulling Image and Creating Container	2
Container Log	3
Checking From Browser	3
Task 2.....	3
Task 3.....	4
Task 4.....	4
System Wide.....	5
User-Specific	5

Task 1

Download Package

The package name is `container-tools`:

```
[root@server ~ #] dnf install -y container-tools
Last metadata expiration check: 0:02:06 ago on T
Dependencies resolved.
```

Registration

Let's register with RedHat account. I have already an account in registry.redhat.io.

So, I will use its credentials here:

```
[root@server ~ #] podman login registry.redhat.io
Username: MrRavan
Password:
Login Succeeded!
[root@server ~ #]
```

I can also register with docker.io:

```
[root@server ~ #] podman login docker.io
Username: mrravan
Password:
Login Succeeded!
[root@server ~ #]
```

Pulling Image and Creating Container

We can use `pull` command to pull the package:

```
[root@server ~ #] podman pull httpd
Resolved "httpd" as an alias (/var/cache/containers/short-name-aliases.conf)
Trying to pull docker.io/library/httpd:latest...
Getting image source signatures
Copying blob 57fb71246055 done
Copying blob 4b705a141cea done
Copying blob 4b31e7d9a3b8 done
Copying blob 4f4fb700ef54 done
Copying blob 6e3e6d1d5b56 done
Copying blob 777279f5ce9f done
Copying config c194ed9b9e done
Writing manifest to image destination
c194ed9b9e8f9c5811690935a41438aea4e16b9dc66a4230b84092b4214f0d9a
[root@server ~ #]
```

We can view:

```
[root@server ~ #] podman images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
docker.io/library/httpd	latest	c194ed9b9e8f	5 days ago	120 MB

```
[root@server ~ #]
```

Let's create a container from the image:

```
[root@server ~ #] mkdir /apache-html
[root@server ~ #] echo '<h2>Hello From Container</h2>' >> /apache-html/index.html
[root@server ~ #] podman run -d --name myapache -v /apache-html:/usr/local/apache2/htdocs:Z -p 8080:80 httpd
1feb7981154c12d98a124f1a69c1f6af33fcbc6ed96616d9137850a86c6be2d6
[root@server ~ #] podman ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
1feb7981154c	docker.io/library/httpd:latest	httpd-foreground	3 seconds ago	Up 3 seconds	0.0.0.0:8080->80/tcp	myapache

```
[root@server ~ #] firewall-cmd --permanent --add-port=8080/tcp
success
[root@server ~ #] firewall-cmd --reload
success
[root@server ~ #]
```

I mounted our container directory (*/usr/local/apache2/htdocs/ - This is default directory in debain/ubuntu for html files*) to /apache-html in VM. Give it myapache as a name.

Moreover, I implemented port forwarding. Mentioned SELinux with :Z. At last I have added port 8080 to firewall.

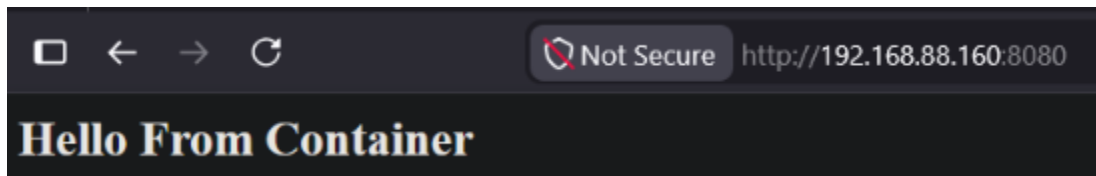
When we check with `podman ps`, it shows container is up.

Container Log

We use `'podman logs'` to see the container logs:

```
[root@server ~ #] podman logs myapache
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 10.88.0.2. Set the 'ServerName' directive globally to suppress this message
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 10.88.0.2. Set the 'ServerName' directive globally to suppress this message
[Thu May 14 15:32:43.074206 2026] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.67 (Unix) configured -- resuming normal operations
[Thu May 14 15:32:43.074705 2026] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGROUND'
[root@server ~ #]
```

Checking From Browser



Task 2

We use `'exec -it'` to switch container's terminal. Here, `'uname -r'` shows kernel version. I have printed `'/etc/os-release'` additionally:

```
[root@server ~ #] podman exec -it myapache /bin/bash
root@1feb7981154c:/usr/local/apache2# uname -r
5.14.0-611.5.1.el9_7.x86_64
root@1feb7981154c:/usr/local/apache2# cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.4
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
root@1feb7981154c:/usr/local/apache2#
```

Task 3

Let's create a volume and pull the image:

```
[root@server ~ #] podman volume create mysql-volume
mysql-volume
[root@server ~ #] podman pull mysql
Trying to pull docker.io/library/mysql:latest...
Getting image source signatures
Copying blob 35300b480a58 skipped: already exists
Copying blob 1dc0d004aaec skipped: already exists
Copying blob 8e3f19ed5e67 skipped: already exists
Copying blob 69edacfaea72 skipped: already exists
Copying blob 4f23ccd26206 skipped: already exists
Copying blob f74943462a4f skipped: already exists
Copying blob 5491a2a13e1b skipped: already exists
Copying blob e80b372994e5 skipped: already exists
Copying blob 802b6dd0c6bd skipped: already exists
Copying blob 8ec50bfa3b7e skipped: already exists
Copying config 094aec6496 done
Writing manifest to image destination
094aec64961bf03283c2c3a4dd8805dc3d61ff83dbc37eaabc30d9d3e379449c
```

Let's create a container and view it:

```
[root@server ~ #] podman run -d \
> --name mydb \
> -e MYSQL_ROOT_PASSWORD=root \
> -e MYSQL_DATABASE=testdb \
> -v myvolume:/var/lib/mysql \
> mysql
5839bd20cf23a988a87dcae05c24b8ce09e6fbddf09ec570a28d5ef421f85ab
[root@server ~ #] podman ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
1feb7981154c	docker.io/library/httpd:latest	httpd-foreground	23 minutes ago	Up 23 minutes	0.0.0.0:8080->80/tcp	myapache
5839bd20cf23	docker.io/library/mysql:latest	mysqld	5 seconds ago	Up 5 seconds	3306/tcp, 33060/tcp	mydb

[root@server ~ #]



MySQL stores its data in /var/lib/mysql inside the container, so that is why I mount here.

Task 4

It is possible to give 2 kinds responses here. First, we can set this service for full system, so, whenever any user logs in, the service will run. This can be created only by root.

Second, we can create a service file for a specific user, so, whenever this specific user logs in, the service will run.

System Wide

```
[root@server ~ #] podman generate systemd --name mydb --files
DEPRECATED command:
It is recommended to use Quadlets for running containers and pods under systemd.

Please refer to podman-systemd.unit(5) for details.
/root/container-mydb.service
[root@server ~ #] mv ./container-mydb.service /etc/systemd/system/
[root@server ~ #] restorecon -v /etc/systemd/system/container-mydb.service
Relabeled /etc/systemd/system/container-mydb.service from unconfined_u:object_r:admin_home_t:s0 to unconfined_u:object_r:systemd_unit_file_t:s0
[root@server ~ #] systemctl daemon-reload
[root@server ~ #] systemctl enable /etc/systemd/system/container-mydb.service
```

Basically I have created service file using `podman generate systemd`. Then I moved it to `/etc/systemd/system/`, so that makes it system wide. I have used `restorecon -v`, because when I moved it to the other place, it did not get correct label, so error occurred. It is good to run `daemon-reload` after each unit creation or modification. At last, I just enabled it.

We can verify that by checking the container status with `podman ps` after the reboot:

```
2026-05-14 21:17:20 /home/mobaxterm ssh root@192.168.88.160
Activate the web console with: systemctl enable --now cockpit.socket

Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Thu May 14 19:03:30 2026 from 192.168.88.1
[root@server ~ #] podman ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5839bd20cf23	docker.io/library/mysql:latest	mysqld	36 minutes ago	Up 9 seconds	3306/tcp, 33060/tcp	mydb

User-Specific

The process is simpler if we create a service user specific.

First, I remove the previous service (system-wide) to prevent confusion:

```
[root@server ~ #] rm -rf /etc/systemd/system/container-mydb.service
[root@server ~ #]
```

Let's remove the container and image to start from scratch:

```
[root@server ~ #] podman stop mydb
mydb
[root@server ~ #] podman rm mydb
mydb
```

```
[root@server ~ #] podman rmi docker.io/library/mysql:latest
Untagged: docker.io/library/mysql:latest
Deleted: 094aec64961bf03283c2c3a4dd8805dc3d61ff83dbc37eaabc30d9d3e379449c
[root@server ~ #]
```

Let's create a new user. Then create a directory for him to use in mounting process. Then let me make him the user and group owner of the directory. Then give write permission to others (recommended). At last, let's use loginctl to allow the process we will do below.

```
[root@server ~ #] useradd Ravan && passwd Ravan
Changing password for user Ravan.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@server ~ #] mkdir /Ravan-Dir
[root@server ~ #] chown Ravan:Ravan /Ravan-Dir
[root@server ~ #] chmod o+w /Ravan-Dir
[root@server ~ #] loginctl enable-linger Ravan
[root@server ~ #]
```

Let's break our ssh connection (using `exit` command) and reconnect as new user:

```
2026-05-14 21:28.03 /home/mobaxterm ssh Ravan@192.168.88.160
Ravan@192.168.88.160's password:
```

In that stage we can `podman login` if it is required as the task 1 above. I do not need it right now.

Let's create a container after pulling image:

```
[Ravan@server ~ $] podman pull mysql
✓ docker.io/library/mysql:latest
Trying to pull docker.io/library/mysql:latest...
Getting image source signatures
Copying blob 35300b480a58 done
Copying blob 1dced004aaec done
Copying blob 8e3f19ed5e67 done
Copying blob 69edacfaea72 done
Copying blob f74943462a4f done
Copying blob 8ec50bfa3b7e done
Copying blob 5491a2a13e1b done
Copying blob e80b372994e5 done
Copying blob 4f23ccd26206 done
Copying blob 802b6dd0c6bd done
Copying config 094aec6496 done
Writing manifest to image destination
094aec64961bf03283c2c3a4dd8805dc3d61ff83dbc37eaabc30d9d3e379449c
[Ravan@server ~ $] podman run -d \
  --name mydb \
  -e MYSQL_ROOT_PASSWORD=root \
  -e MYSQL_DATABASE=testdb \
  -v /Ravan-Dir:/var/lib/mysql:Z \
  mysql
bcb4a8010de002ee8f57dc0e78ddf492ac622dd1b84280b1f63e072aa3770211
```

Let's create a directory for the service file and switch to there. Then create a service file using `podman generate` command:

```
[Ravan@server ~]$ mkdir -p ~/.config/systemd/user
[Ravan@server ~]$ cd ~/.config/systemd/user/
[Ravan@server ~/.config/systemd/user]$ podman generate systemd --name mydb --files

DEPRECATED command:
It is recommended to use Quadlets for running containers and pods under systemd.

Please refer to podman-systemd.unit(5) for details.
/home/Ravan/.config/systemd/user/container-mydb.service
```

It is good to use `daemon-reload` after creating a new unit. We should enable the service file to make it work properly after the reboot:

```
[Ravan@server ~/.config/systemd/user]$ systemctl --user daemon-reload
[Ravan@server ~/.config/systemd/user]$ systemctl --user enable container-mydb.service
Created symlink /home/Ravan/.config/systemd/user/default.target.wants/container-mydb.service → /home/Ravan/.config/systemd/user/container-mydb.service.
```

Let's check it. So, when we connect with ssh as Ravan user, the container should be up:

```
2026-05-14 21:38:41 /home/mobaxterm ssh Ravan@192.168.88.160
Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Thu May 14 21:28:27 2026 from 192.168.88.1
[Ravan@server ~]$ podman ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
bcb4a8010de0   docker.io/library/mysql:latest      mysqlqld                 4 minutes ago Up 4 minutes   3306/tcp, 33060/tcp              mydb
[Ravan@server ~]$
```

Moreover, it should not work when we log in as another user like root:

```
2026-05-14 21:39:39 /home/mobaxterm ssh root@192.168.88.160
Activate the web console with: systemctl enable --now cockpit.socket

Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Thu May 14 21:29:17 2026 from 192.168.88.1
[root@server ~]# podman ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
[root@server ~]#
```